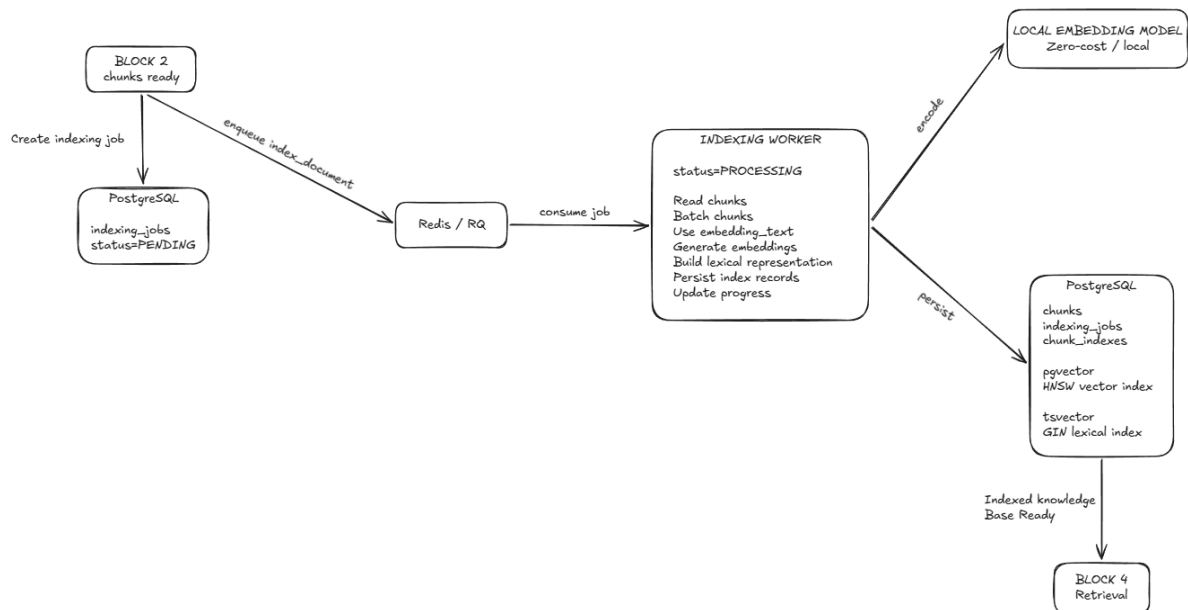


BLOCK 3



Block 2 → PostgreSQL: khi **chunks** đã sẵn sàng, tạo một **indexing_job** với trạng thái ban đầu **PENDING**.

Block 2 → Redis/RQ: enqueue job **INDEX_DOCUMENT** chứa **indexing_job_id**, **document_id**, **request_id**.

Redis/RQ → Indexing Worker: Worker consume job và bắt đầu quá trình indexing.

Indexing Worker → PostgreSQL: đọc các **chunks** của document theo thứ tự **chunk_index**.

Indexing Worker xử lý nội bộ:

- gom chunks thành batch;
- lấy **embedding_text** để tạo dense embedding;
- lấy **content_text** để tạo lexical representation.

Indexing Worker → Local Embedding Model: gửi batch **embedding_text** để encode.

Local Embedding Model → Indexing Worker: trả về các dense vector.

Indexing Worker → PostgreSQL / pgvector: persist kết quả indexing vào **chunk_indexes**.

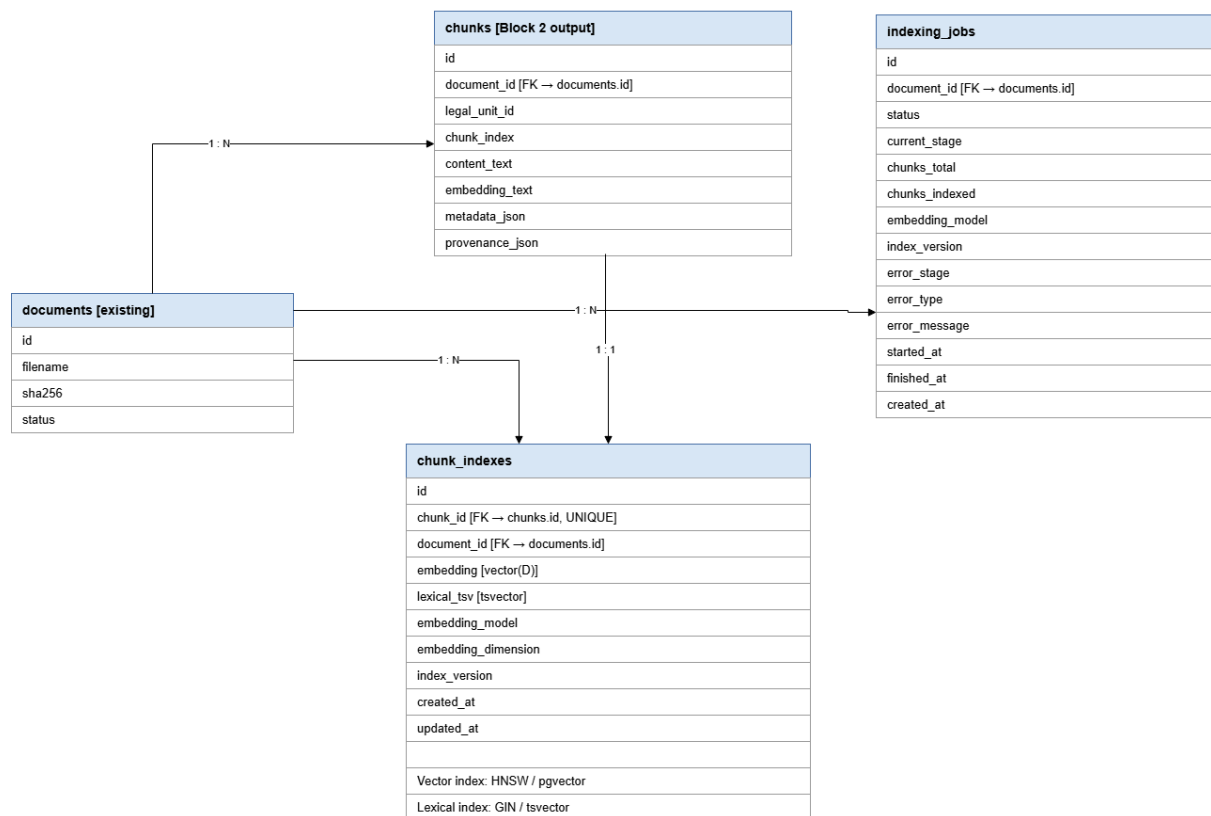
PostgreSQL duy trì:

- **chunks**;
- **indexing_jobs**;
- **chunk_indexes**;
- **pgvector** + **HNSW** cho dense search;
- **tsvector** + **GIN** cho lexical search.

Khi toàn bộ chunk đã index thành công, Worker cập nhật **indexing_job.status = COMPLETED**.

PostgreSQL → Block 4: dense + lexical indexes đã sẵn sàng để Block 4 Retrieval sử dụng.

BLOCK 3 — POSTGRESQL + VECTOR DATA CONTRACT



documents

Entity gốc đã tồn tại từ Block 1. Block 3 không thay đổi logic của bảng này.

chunks Đây là **output của Block 2** và input chính của Block 3.

Trong đó:

- **embedding_text** → dense embedding.
 - **content_text** → lexical representation.
 - metadata/provenance vẫn giữ nguyên ở **chunks**, không duplicate vào index nếu không cần.
-

indexing_jobs Theo dõi lifecycle riêng của Block 3.

Ví dụ:

```
status = PROCESSING
current_stage = EMBEDDING
```

```
chunks_total = 76
chunks_indexed = 32
```

Nhờ vậy ta biết indexing đang đi đến đâu.

chunk_indexes Đây là derived searchable representation của mỗi chunk.

Dense side:

embedding vector(D)

↓

pgvector

↓

HNSW

Lexical side:

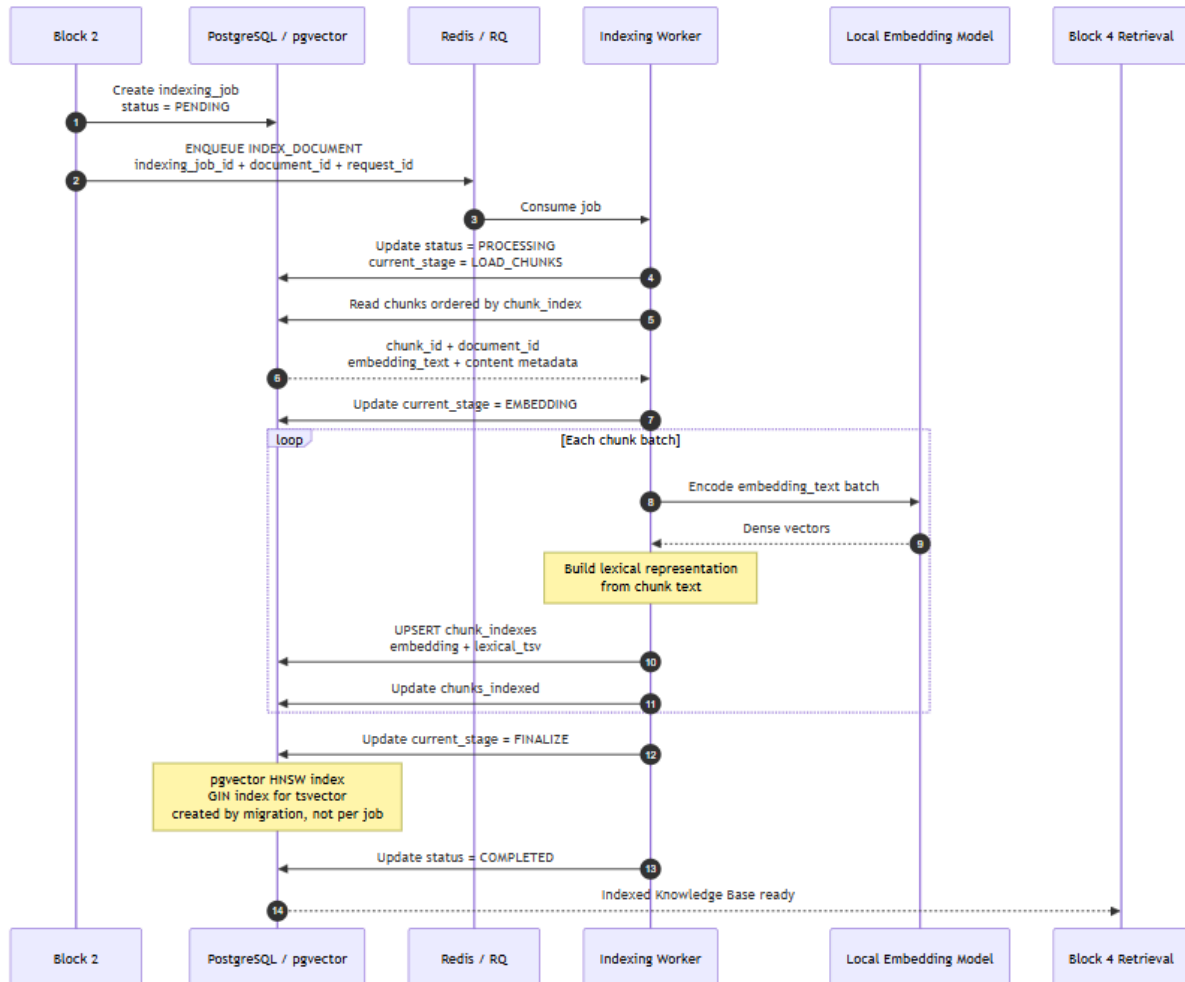
lexical_tsv

↓

PostgreSQL tsvector

↓

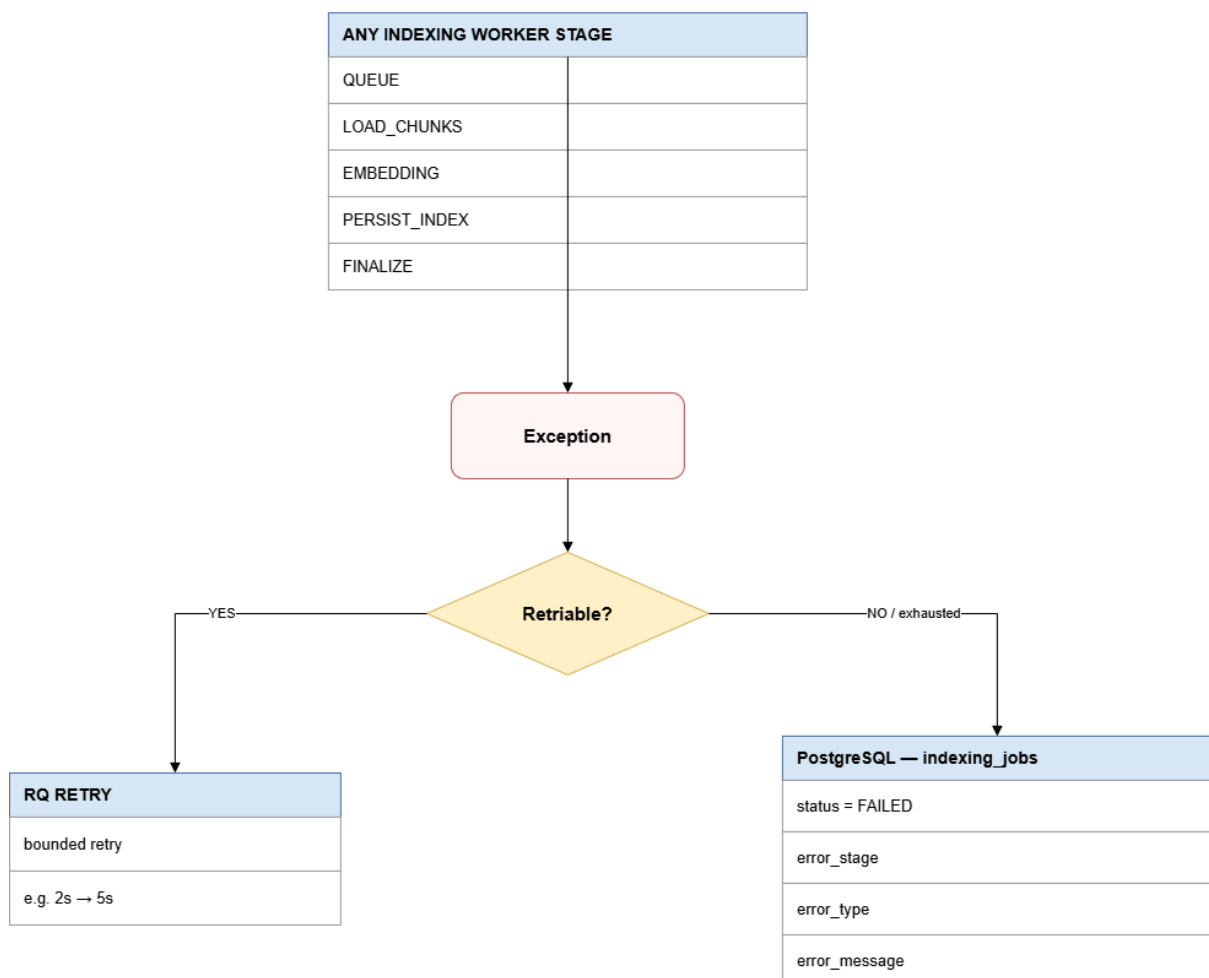
GIN



- **Block 2 → PostgreSQL:** tạo `indexing_job` với `status = PENDING`, có thể đặt `current_stage = QUEUE`.
- **Block 2 → Redis/RQ:** enqueue `INDEX_DOCUMENT` chứa `indexing_job_id`, `document_id`, `request_id`.
- **Redis/RQ → Indexing Worker:** Worker consume job.
- **Worker → PostgreSQL:** cập nhật `status = PROCESSING`, `current_stage = LOAD_CHUNKS`.
- **Worker → PostgreSQL:** đọc các `chunks` theo `chunk_index`.
- **PostgreSQL → Worker:** trả `chunk_id`, `document_id`, `content_text`, `embedding_text` và metadata cần thiết.
- **Worker → PostgreSQL:** cập nhật `current_stage = EMBEDDING`.
- **Worker xử lý nội bộ:** gom chunks thành từng batch.
- **Worker → Local Embedding Model:** encode một batch `embedding_text`.
- **Local Embedding Model → Worker:** trả dense vectors.

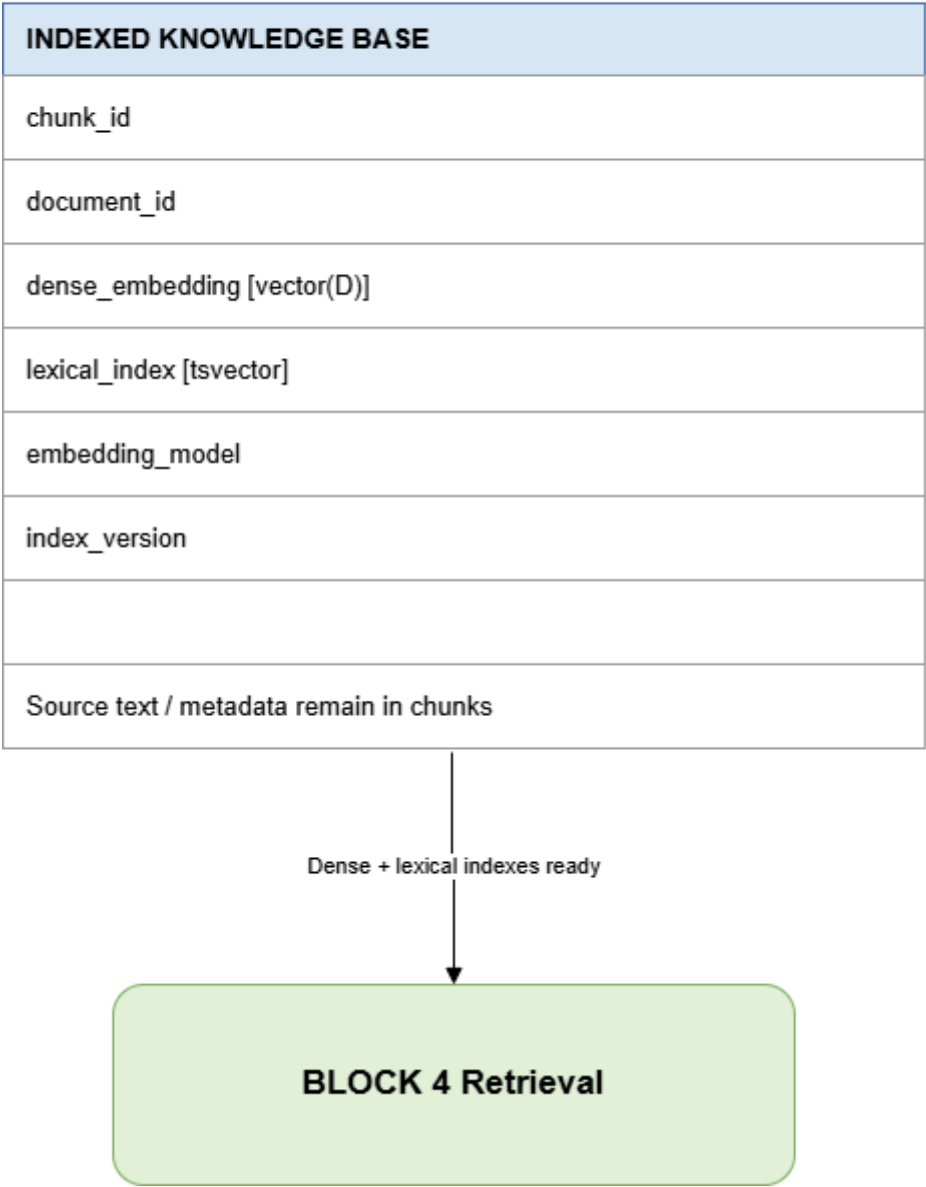
- **Worker xử lý nội bộ:** tạo lexical representation từ `content_text`.
- **Worker → PostgreSQL:** trước khi ghi index, cập nhật `current_stage = PERSIST_INDEX`.
- **Worker → PostgreSQL:** UPSERT `chunk_indexes` gồm `embedding + lexical_tsv`.
- **Worker → PostgreSQL:** cập nhật `chunks_indexed`.
- Quy trình encode + persist lặp lại cho đến khi toàn bộ chunks hoàn tất.
- **Worker → PostgreSQL:** cập nhật `current_stage = FINALIZE`.
- **Worker → PostgreSQL:** khi toàn bộ indexing thành công, đổi `status = COMPLETED`.
- **PostgreSQL → Block 4:** Indexed Knowledge Base đã sẵn sàng để Retrieval sử dụng.

BLOCK 3 — FAILURE / RETRY LANE



Failure Lane giúp biết **Block 3 fail ở queue, loading, embedding hay persistence**, thay vì chỉ thấy một trạng thái **FAILED** chung chung.

BLOCK 3 — OUTPUT CONTRACT



Hình này trả lời:

Block 3 cam kết giao thứ gì cho Block 4?

Output là: *INDEXED KNOWLEDGE BASE*

gồm searchable representation:

chunk_id

document_id

dense_embedding

lexical_index

embedding_model

index_version

dense_embedding

Vector biểu diễn semantic meaning của chunk:

embedding_text -> Embedding Model -> vector(D)

Block 4 dùng nó cho semantic/vector search.

lexical_index

Representation phục vụ keyword/full-text retrieval:

content_text -> tsvector -> GIN

Block 4 dùng nó cho lexical search.

embedding_model

Cho biết vector được tạo bằng model nào.

Cần thiết vì: Vector sinh bởi model A không nên bị coi tương đương vector sinh bởi model B.

index_version

Cho biết index thuộc version pipeline nào.

Sau này nếu đổi:

embedding model
chunk indexing strategy
lexical strategy

ta có thể nhận biết index cũ/mới.

Source text và metadata không bị copy

Chúng vẫn nằm trong:

chunks

Nên Block 4 có thể:

chunk_indexes
|
| *chunk_id*
▼
chunks

để lấy lại:

content_text
metadata_json
provenance_json

sau khi retrieval tìm được các chunk phù hợp.

Điểm cần nhớ: **Block 3 không tạo knowledge mới. Nó biến output Block 2 thành dữ liệu có thể search hiệu quả.**